API Design: Learning From Mistakes in the C Library

The C library is perhaps the most widely reused library in the world. However, it has numerous flaws and inconsistencies, from which we can learn.

Design is hard, and API design is harder. This is because applications and systems are designed to be used, while in the case of APIs, we design them not just to be used but also reused. Applications are designed for end customers to use; however, with the APIs in libraries, platforms and frameworks, the end user is another programmer who wants to get the work done via the API. There are more things to consider in API design than with application design. For example, compatibility issues (such as backward compatibility) are an additional concern for an API designer. In frameworks and libraries, fixing a bug is difficult when compared to fixing the same bug in a stand-alone application. For example, if the bug fix involves a change to the public interface exposed by the API, then the existing applications written based on that interface will fail. Hence, the evolution of an API is harder than evolving applications.

C is used in a wide range of devices, ranging from microwave ovens to air-planes. With the popularity of the C language and its widespread use, APIs in the C library have been used by millions of programmers worldwide over the last four decades. The C library has had a huge impact on the design of libraries in languages that arrived later. For example, the 'format string' approach is not type-safe, but for formatted output, it is a very convenient approach. The C++ language provides a type-safe alternative to formatted output, but that is not as convenient as the C 'format style' approach to use. The Java language's *println* approach is type-safe, but is not convenient. Hence, Java introduced *printf* with the 'format string' approach in its library. Similarly, C# has format strings, and only its format specifier is slightly different; instead of the % symbol, it uses {} symbols. This is just one example of the influence of C library design on languages that came up later.

Given the influence and widespread use it enjoyed, it is surprising to find that the C library has numerous flaws and mistakes! So, it would be interesting for us to understand these mistakes and learn from them, so that we can avoid them in future.

Subtle differences in similar-looking functions

In API design, it's important to keep similar things to be similar. All of us are familiar with the *puts()* function, which writes the string passed as an argument to the standard output, followed by a newline character. Now, if you compare *puts()* with the similar looking *fputs()* function, you will find a slight difference in semantics: *fputs()* writes the string passed as an argument to the given stream *without* the newline character. Now, this difference is subtle, and as programmers it's difficult for us to remember it every time we use these functions.

Inconsistent argument order in functions

Similar functions must have a similar argument type/order and return type. Consider the following two functions:

```
void bcopy(const void *src, void *dst, size_t n);
void *memcpy(void *dst, const void *src, size_t n);
```

Note the order of the first two arguments for these. As you can see, the order is interchanged. As a programmer, you should remember which the source string is and which the destination string is, because if you get confused, you may introduce a subtle bug by copying the string from the destination to the source! That is why consistency is an important factor in API design, and many C library functions are inconsistent.

Unsafe functions

A considerable number of C functions are unsafe. For example, the functions are often not type-safe, or they are not safe to call. For example, functions like *scanf()* and *gets()* read the string to the input buffer without checking the size. If the string read is larger than the buffer size, it results in a buffer overflow. For this reason, these C functions are infamous for buffer overflow attacks, which lead to security exploits.

Stateful functions

Library functions are supposed to not remember state. Functions such as *strtok* are stateful—they remember the state based on the